

Room - OWASP Top 10 2025: Insecure Data Handling

This room will introduce you to 3 elements of the [OWASP](#) Top 10 list (2025). In this room, you will learn about the elements relating to application behaviour and user input. We will cover these vulnerabilities briefly, how to prevent them, and finally, you will practice exploiting these vulnerabilities:

- A04: Cryptographic Failures
- A05: Injection
- A08: Software or Data [Integrity](#) Failures

Cryptographic Failures occur when sensitive data is not properly protected because of weak encryption, poor implementation, or insecure handling of secrets.

Common examples:

- Storing passwords without hashing
- Using weak/outdated algorithms like MD5, SHA1, or DES
- Exposing encryption keys in code or configuration files
- Failing to encrypt data during transmission
- Creating custom ("roll your own") cryptography instead of using trusted standards

Why it matters:

- Attackers can steal passwords, decrypt sensitive data, or impersonate users.
- Weak cryptography can completely break application security.

How to prevent cryptographic failures:

- Use strong modern algorithms and libraries
- Hash passwords with secure hashing functions such as:
 - bcrypt
 - scrypt
 - Argon2
- Use trusted encryption standards instead of custom algorithms
- Store secrets in secure key management systems, not in source code or repositories
- Protect data both at rest and in transit

Practical challenge summary:

- The web app at [10.48.178.155:8001](#) is a note-sharing service.
- It uses a weak shared derivative key for note protection.

- Your task is to exploit this weakness, unlock all notes, and retrieve the flag.

Core lesson:

- Cryptography is only secure when implemented correctly. Weak algorithms, exposed secrets, and shared keys can make encryption useless even if encryption exists.

Injection vulnerabilities happen when an application takes untrusted user input and passes it directly into interpreters, databases, shells, or templates without proper validation or sanitization.

Why it matters:

- Attackers can execute unintended commands or queries.
- Injection can lead to data theft, remote code execution, authentication bypass, or complete system compromise.
- Injection remains one of the most dangerous and common web vulnerabilities.

Common types of injection:

- SQL Injection
- Command Injection
- AI Prompt Injection
- Server-Side Template Injection (SSTI)

Example:

- In SQL Injection, user input is directly added into a database query without protection, allowing attackers to manipulate the query.

How to prevent injection:

- Treat all user input as untrusted
- Use parameterized queries/prepared statements for databases
- Avoid passing user input directly to system shells
- Use safe APIs instead of shell execution
- Validate input types and formats strictly
- Sanitize and escape dangerous characters

Practical challenge summary:

- The challenge demonstrates Server-Side Template Injection (SSTI).
- The vulnerable app dynamically renders user-controlled content.
- Your goal is to abuse the template rendering system to retrieve a flag stored on the server.
- The challenge is hosted at `10.48.178.155:8000`.

Core lesson:

- Injection vulnerabilities occur when applications trust user input too much. Proper validation, sanitization, and safe coding practices are essential to prevent attackers from executing unintended actions.

Software or Data Integrity Failures happen when applications trust software, updates, configurations, or data without verifying their authenticity or integrity.

Why it matters:

- Attackers can tamper with software, updates, or serialized data to execute malicious actions.
- If applications blindly trust external or modified data, attackers may gain code execution or compromise the system.

Common examples:

- Installing updates without verification
- Loading scripts or files from untrusted sources
- Accepting modified configuration/data files
- Insecure deserialization attacks
- Weak CI/CD security and build pipeline trust

How to prevent these failures:

- Verify software and update integrity using cryptographic checksums/signatures
- Only trust verified sources
- Protect CI/CD pipelines and build systems
- Validate all critical data before processing
- Enforce strict trust boundaries
- Monitor and secure dependencies and artifacts

Practical challenge summary:

- The challenge demonstrates an Insecure Deserialisation vulnerability in Python.
- Deserialization becomes dangerous when untrusted serialized data is processed without validation.
- Attackers can craft malicious serialized input to execute unintended code or manipulate application behavior.
- The practical is hosted at 10.48.178.155:8002.

Core lesson:

- Applications should never blindly trust code, updates, or serialized data. Integrity verification and strict validation are essential to prevent tampering and malicious payload execution.